LOGO



5 工厂方法模式

软件学院

目录
CONTENTS

5 工厂方法模式

5.1 模式动机与定义

5.2 模式结构与分析

5.3 模式实例与解析

5.4 模式效果与应用



工厂方法模式重要等级： ★★★★★

工厂方法模式难度等级： ★★☆☆☆

本章难点：

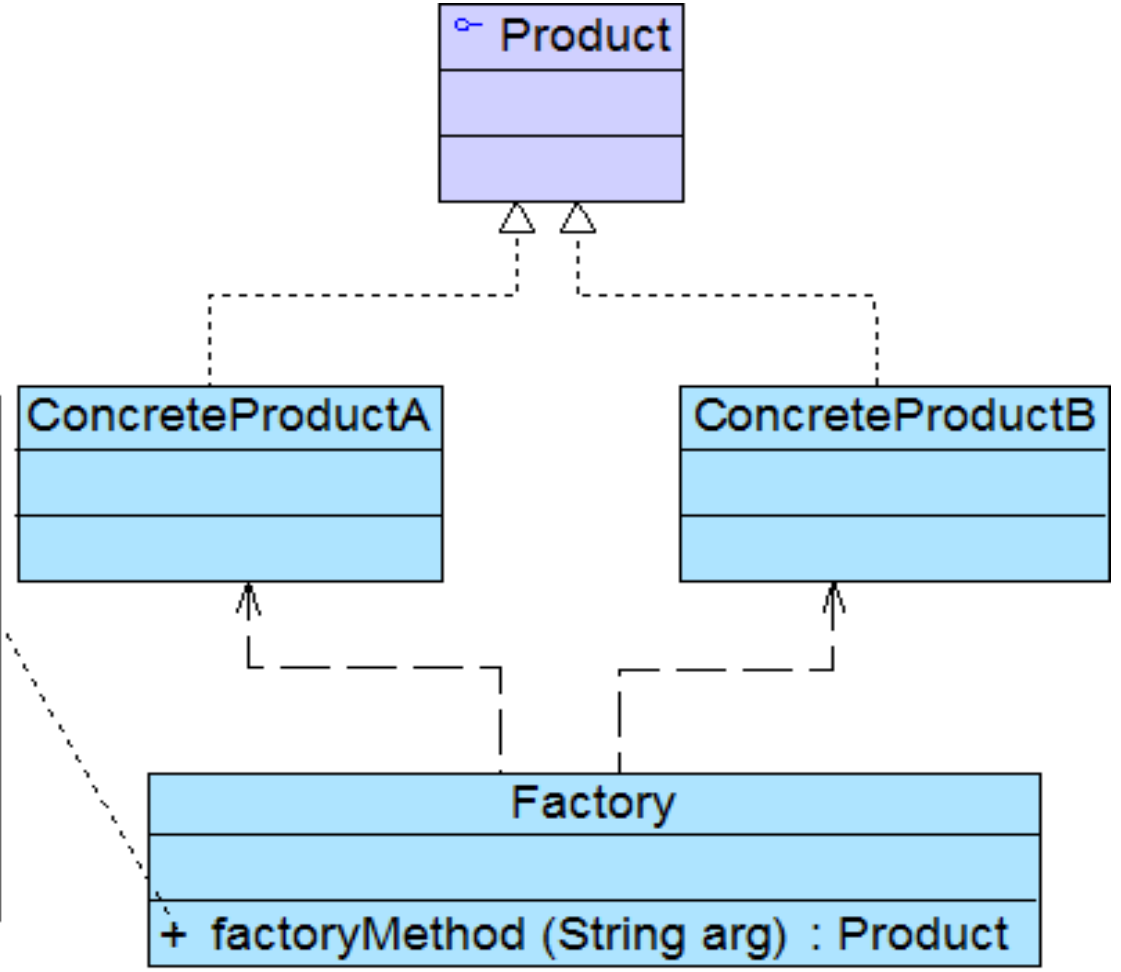
1、理解引入抽象工厂类的原因

2、工厂方法中多态性的体现和客户端代码实现

3、通过DOM和Java反射机制来操作XML配置文件

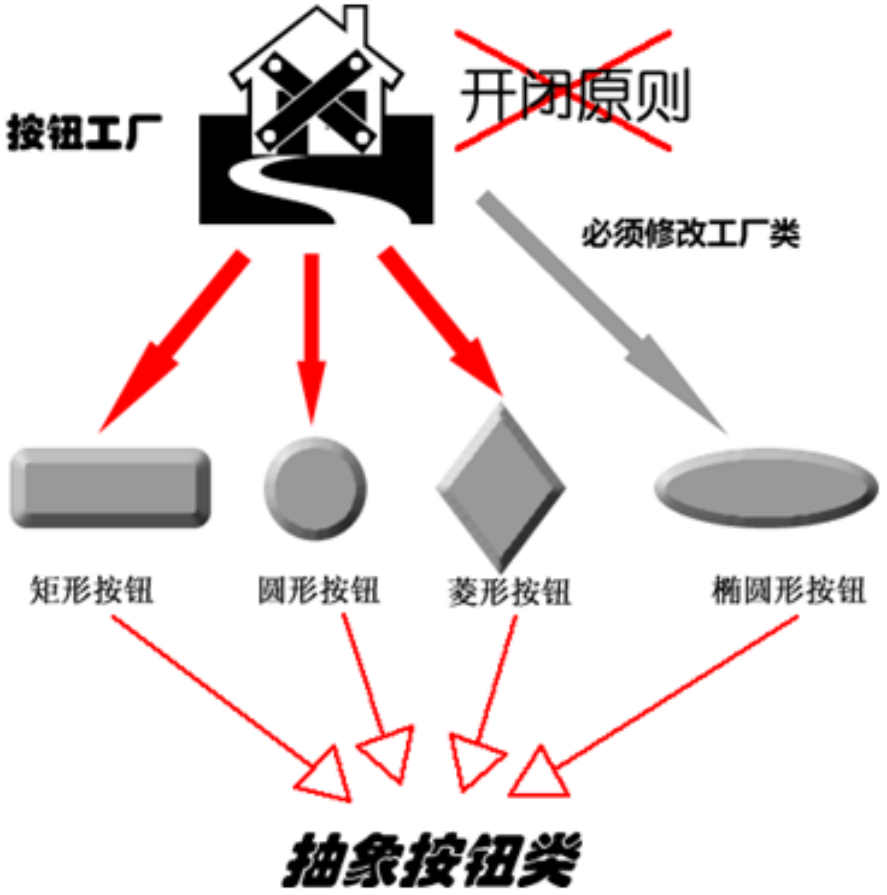
回顾简单工厂设计模式的不足（加入新产品时，必须修改工厂类）

```
if(arg.equalsIgnoreCase("A"))
{
    return new ConcreteProductA();
}
else if(arg.equalsIgnoreCase("B"))
{
    return new ConcreteProductB();
}
else
{
    ......
}
```

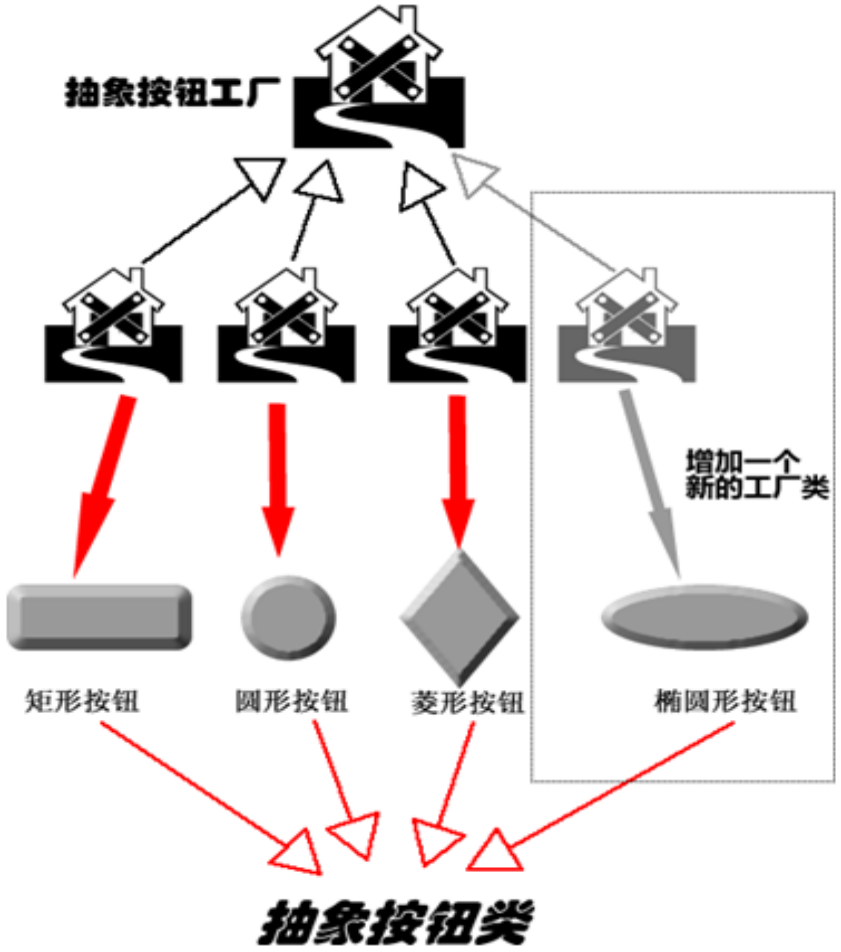


5.1 模式动机与定义

- 模式动机



简单工厂模式



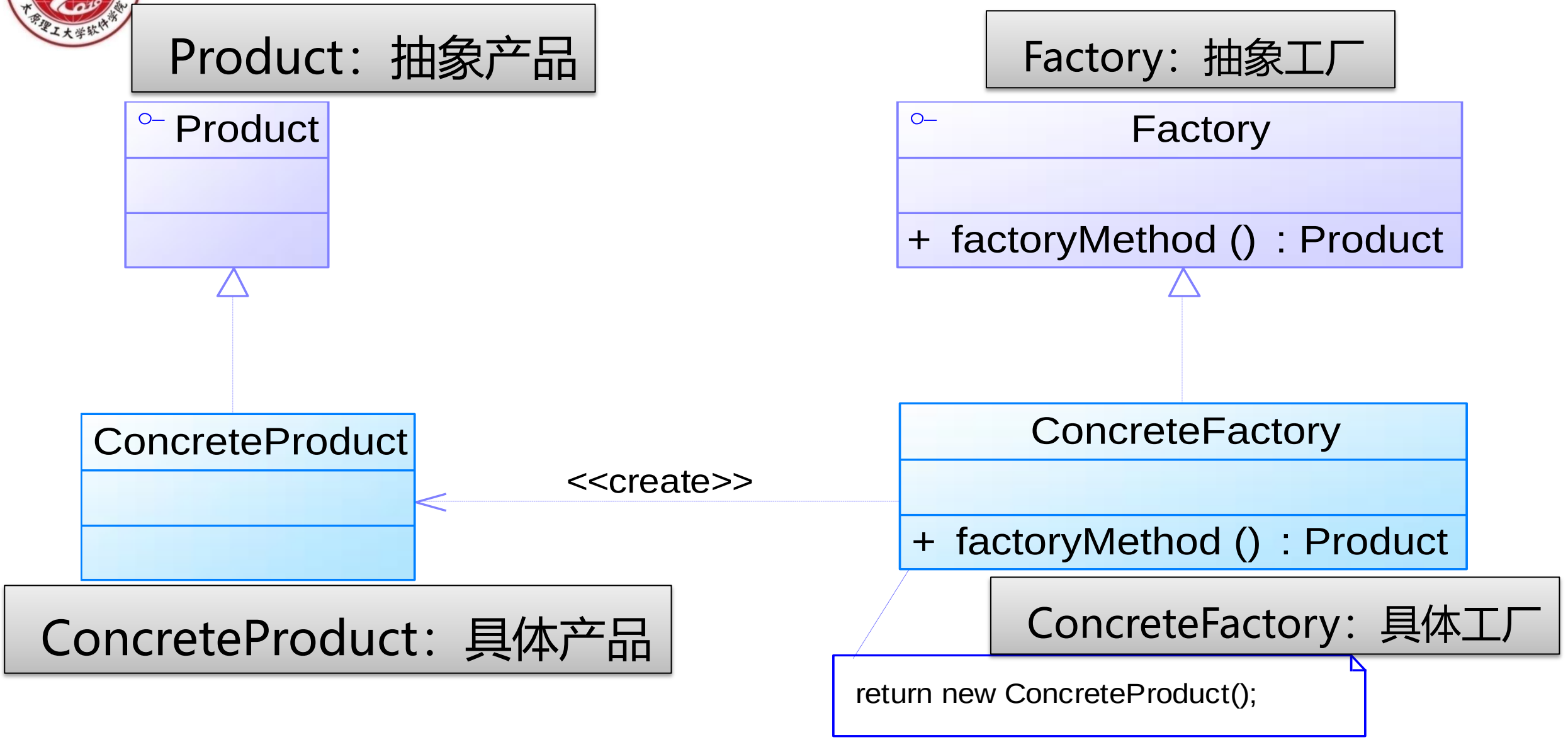
工厂方法模式



5.1 模式动机与定义

- **模式定义**
  - ✓ 工厂方法模式(Factory Method Pattern)简称工厂模式，也叫虚拟构造器(Virtual Constructor)模式或者多态工厂(Polymorphic Factory)模式，它属于类创建型模式。
  - ✓ 在工厂方法模式中，工厂父类负责定义创建产品对象的公共接口，而工厂子类则负责生成具体的产品对象，这样做的目的是将产品类的实例化操作延迟到工厂子类中完成，即通过工厂子类来确定究竟应该实例化哪一个具体产品类。

模式结构


Product：抽象产品
Factory：抽象工厂
Product
Factory
+ factoryMethod () : Product
ConcreteProduct
<<create>>
ConcreteFactory
+ factoryMethod () : Product
ConcreteProduct：具体产品
ConcreteFactory：具体工厂
return new ConcreteProduct();

- 模式分析（还以第4章中多种支付方式为例）

抽象产品类

```
public abstract class AbstractPay
{
    public abstract void pay();
}
```

具体产品类1

```
public class CashPay extends AbstractPay{
    public void pay(){
        //现金支付方式处理代码    }
}
```

具体产品类2

```
public class CreditcardPay extends AbstractPay{
    public void pay(){
        //信用卡支付方式处理代码 }
}
```

每一种支付方式封装在一个单独的具体类中，相对独立，修改其一对其他类无影响

- 模式分析（还以第4章中多种支付方式为例）

```
public class PayMethodFactory
{
    public static AbstractPay getPayMethod(String type)
    {
        if(type.equalsIgnoreCase(“cash”)){
                return new CashPay();//根据参数创建具体产品
      }
       else if(type.equalsIgnoreCase(“creditcard”)) {
                return new CreditcardPay();//根据参数创建具体产品

      }
        …
}
```

工厂类

抽象化、层次类

- 模式分析（还以第4章中多种支付方式为例）

```
public abstract class PayMethodFactory
{
    public abstract AbstractPay getPayMethod( );
}
```

抽象
工厂类

```
public class CashPayFactory extends  PayMethodFactory
{
    public AbstractPay getPayMethod( )
    {
        return new CashPay();
      }
}
```

具体
工厂类之一

- 模式分析（还以第4章中多种支付方式为例）

```
PayMethodFactory factory;
AbstractPay payMethod;
factory=new CashPayFactory();
payMethod=factory.getPayMethod();
payMethod.pay();
```

客户类

具体工厂类的实例化过程，可以不用new，而是将具体类的类名写入配置文件中，再通过Java的反射机制，读取XML配置文件，根据配置文件中存储的类名字符串生成对象。

- 模式分析（还以第4章中多种支付方式为例）

```
<?xml version="1.0"?>
<config>
   <className>CashPayFactory</className>
</config>
```

配置文件

具体工厂类的实例化过程，可以不用new，而是将具体类的类名写入配置文件中，再通过Java的**反射机制**，读取XML配置文件，根据配置文件中存储的类名字符串生成对象。



Java反射机制（Java Reflection）

- 程序运行时获取已知名称的类或已有对象的相关信息的一种机制，包括类的方法、属性、超类等信息，还包括实例的创建和实例类型的判断等。
- 反射中使用最多的类是Class，Class类的实例表示正在运行的Java应用程序的类和接口，其forName(String className)方法可以返回与带有给定字符串名的类或接口相关联的Class对象，再通过Class对象的newInstance()方法创建此对象所表示的类的一个新实例，即通过一个类名字符串得到类的实例。

```
//创建一个字符串类型的对象
Class c = Class.forName("String");
Object obj = c.newInstance();
return obj;
```



5.2 模式结构与分析

- Java反射机制（Java Reflection）

- 工具类XMLUtil代码片段

```
Public static Object geBean(){
    DocumentBuilderFactory dFactory = DocumentBuilderFactory.newInstance();
    DocumentBuilder builder = dFactory.newDocumentBuilder();
    Document doc;
    doc = builder.parse(new File("config.xml"));

     NodeList nl = doc.getElementsByTagName("className");
     Node classNode=nl.item(0).getFirstChild();
     String cName=classNode.getNodeValue();

    //通过类名生成实例对象并将其返回
     Class c=Class.forName(cName);
     Object obj=c.newInstance();
     return obj;
}
```



5.2 模式结构与分析

- 模式分析（还以第4章中多种支付方式为例）

```
PayMethodFactory factory;
AbstractPay payMethod;
factory =(PayMethodFactory)XMLUtil.getBean();
payMethod=factory.getPayMethod();
payMethod.pay();
```

客户类代码修改



模式分析

- ✓ 工厂方法模式是简单工厂模式的进一步抽象和推广
- ✓ 工厂方法模式保持了简单工厂模式的优点，并克服了它的缺点
- ✓ 核心的工厂类不再负责所有产品的创建，而是将具体创建工作交给其子类去完成
- ✓ 可以允许系统在不修改工厂角色的情况下引进新产品
- ✓ 增加具体产品-->增加具体工厂，符合“开闭原则”



5.3 模式实例与解析

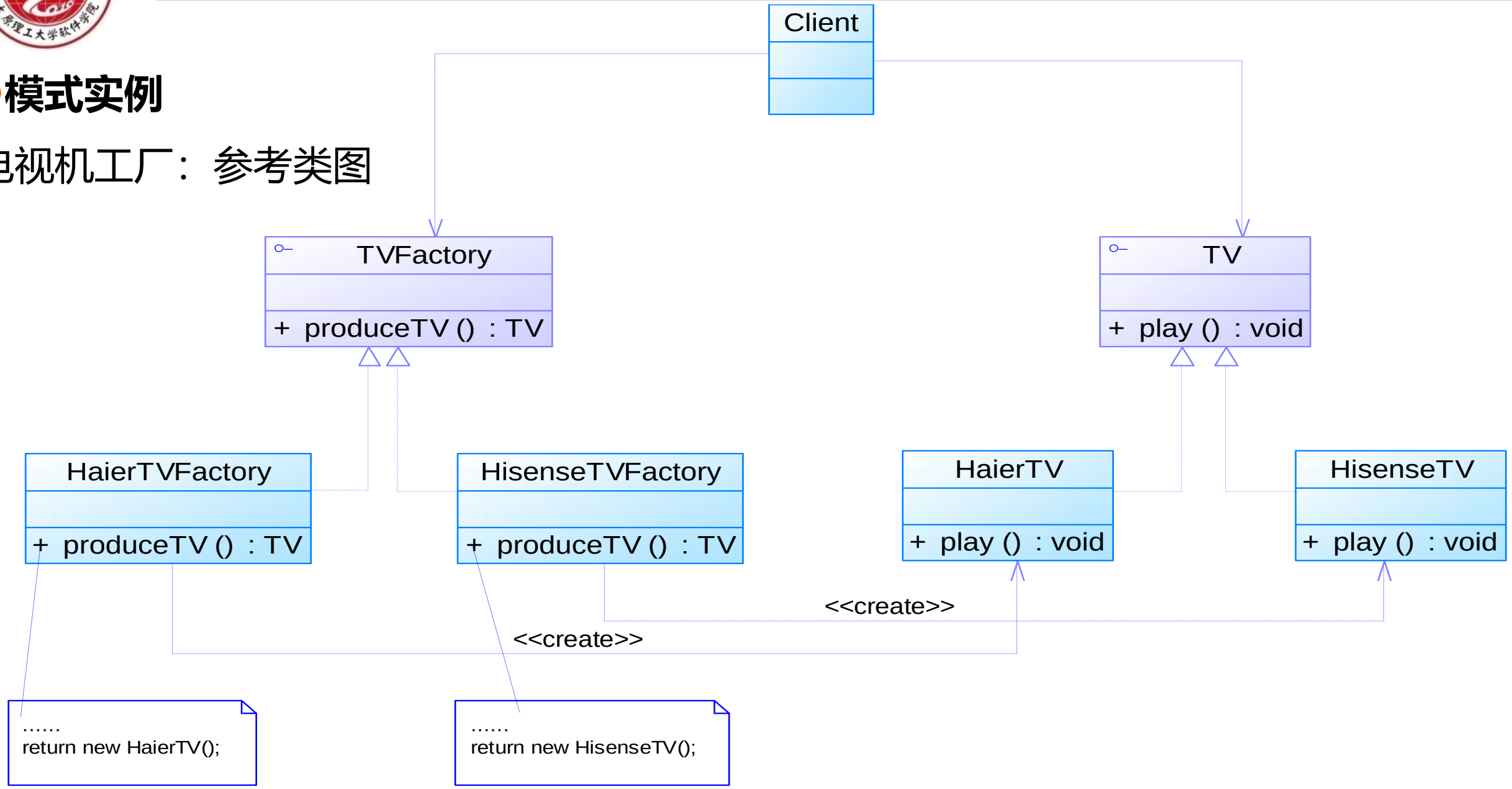
✓ 电视机工厂：实例说明

- 将原有的电视机工厂进行分割，为每种品牌的电视机提供一个子工厂，海尔工厂专门负责生产海尔电视机，海信工厂专门负责生产海信电视机，如果需要生产TCL电视机或创维电视机，只需要对应增加一个新的TCL工厂或创维工厂即可，原有的工厂无须做任何修改，使得整个系统具有更加的灵活性和可扩展性。

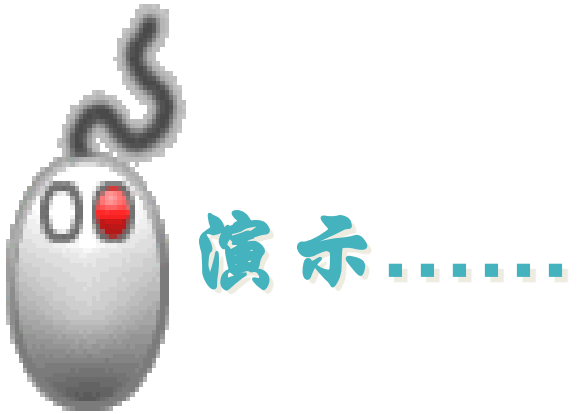
5.3 模式实例与解析

●模式实例

电视机工厂：参考类图



- **模式实例**
  - ✓电视机工厂：参考代码
    - DesignPatterns之factorymethod包





5.3 模式实例与解析

辅助代码：

- 工具类-XMLUtilTV
- <?xml version="1.0"?>

  <config>

  <className>HaierTVFactory</className>

  </config>
- 抽象写在代码中，具体写在配置中。符合开闭原则和依赖倒转原则。
- 用户测试类Client
- 扩展生产TCL电视？增加具体产品子类TCLTV，增加具体工厂类TCLFactory



5.4 模式效果与应用

- **工厂方法模式优点：**
  - ✓ 工厂方法用来创建客户所需要的产品，同时还向客户隐藏了哪种具体产品类将被实例化这一细节
  - ✓ 能够让工厂自主确定创建何种产品对象，而如何创建这个对象的细节则完全封装在具体工厂内部
  - ✓ 在系统中加入新产品时，完全符合开闭原则



5.4 模式效果与应用

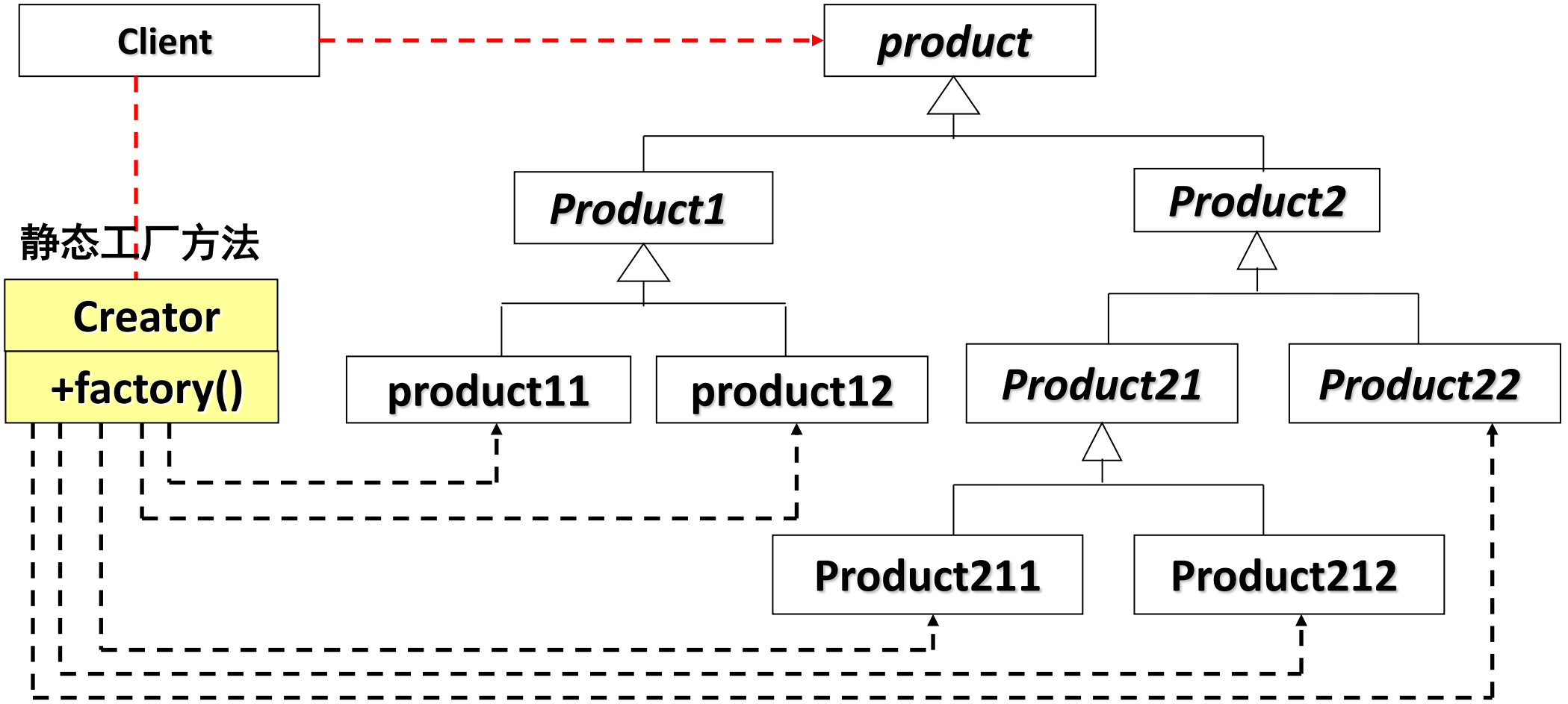
- **工厂方法模式缺点：**
  - ✓ 系统中类的个数将成对增加，在一定程度上增加了系统的复杂度，会给系统带来一些额外的开销
  - ✓ 增加了系统的抽象性和理解难度



5.4 模式效果与应用

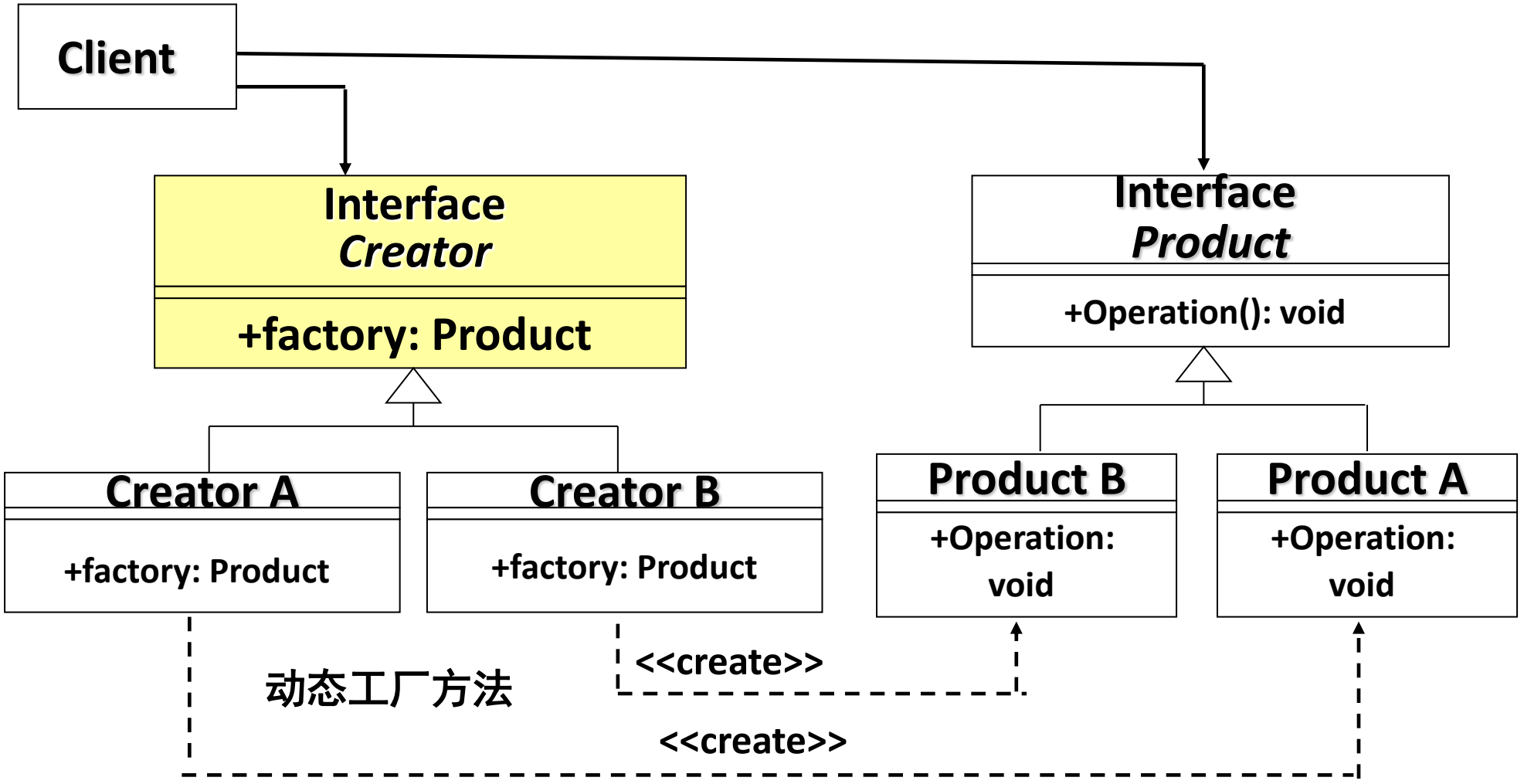
- **在以下情况下可以使用工厂方法模式：**
  - ✓客户端不知道它所需要的对象的类（客户端不需要知道具体产品类的类名，只需要知道所对应的工厂即可，具体产品对象由具体工厂类创建）
  - ✓抽象工厂类通过其子类来指定创建哪个对象

简单工厂模式和工厂方法模式的对比



Simple Factory method pattern

简单工厂模式和工厂方法模式的对比



Factory method pattern



简单工厂模式和工厂方法模式的对比

相同：两个工厂方法返回类型都是超类类型

不同：a)中心不同

工厂方法模式的核心是抽象工厂类（接口），简单工厂模式的核心是具体工厂类

b)工厂方法不同，一个静态，一个动态

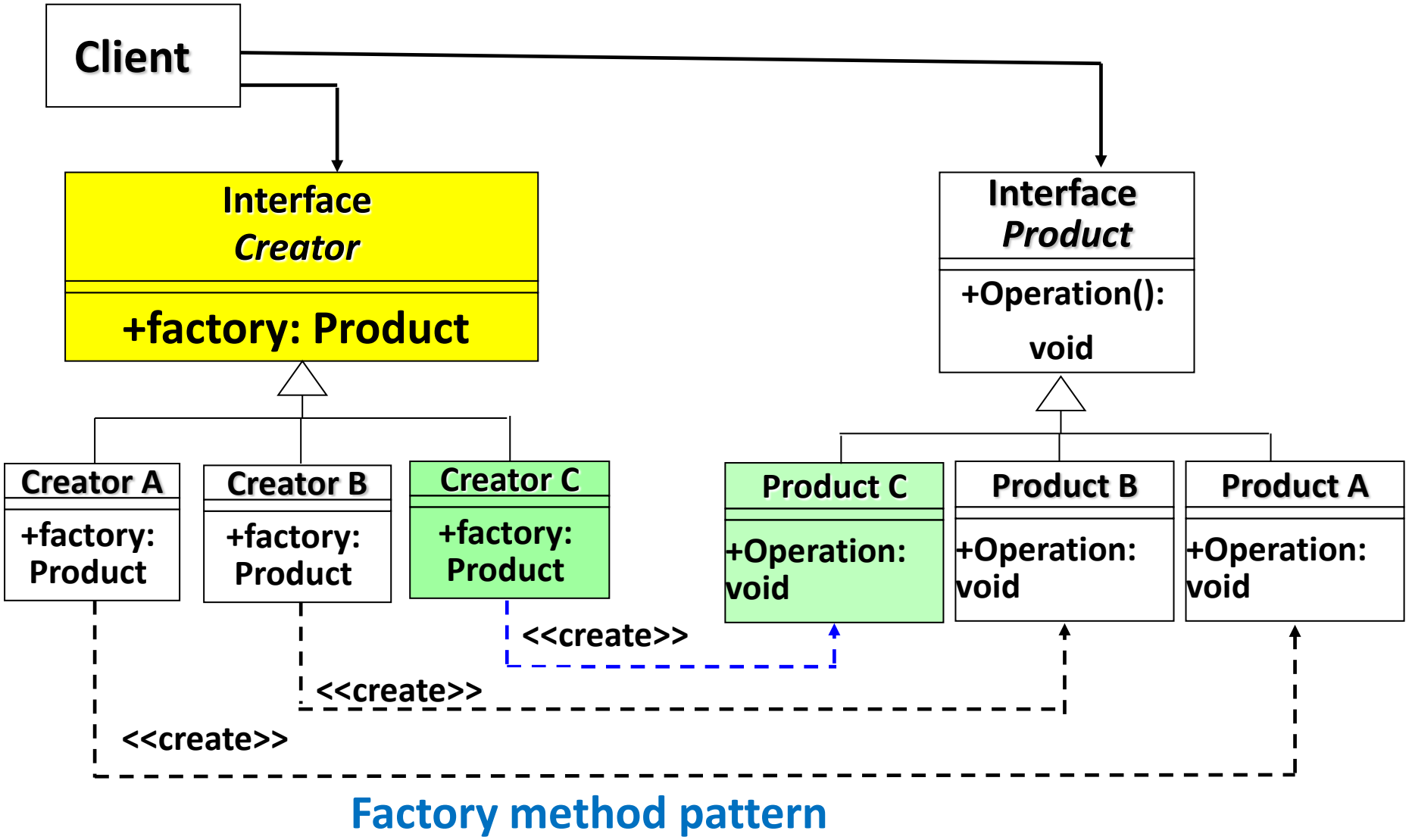
简单工厂模式中，工厂类是静态类，工厂方法模式中，工厂方法是动态的，并分布在具体工厂类中

c) 工厂方法模式支持开闭原则，但是简单工厂方法模式不支持开闭原则

c)

简单工厂模式和工厂方法模式的对比

不同：d)工厂方法模式的可扩展性.



Factory method pattern



图片来源：Veer图库 www.veer.com